Part A: Object-Oriented Design

Design Principles



Abu Saleh Musa Miah (PhD)

Presentation Outline

- ✓ **Design Principle**
- ✓ Evaluating Design Complexity

Design Principles

- ✓ Provides **guidelines for evaluating software structure**
- ✓ Ensures software is **flexible, reusable, and maintainable**
- ✓ Covers **behavior modeling** using:
  - ➢ **UML State Diagrams**
  - ➢ **UML Sequence Diagrams**

◆ Evaluating Design Complexity

- ✓ **Keep modules simple** to avoid unnecessary complexity
- ✓ Excessive complexity → **harder to understand** → more **bugs**
- ✓ **Evaluation of design complexity** is essential for quality assurance
- ✓ Complexity applies to:
  - ➢ **Classes**
  - ➢ **Methods** within classes
- ✓ In this module, **"module" = program unit containing classes and methods**





Design Principles

1. WHAT ARE DESIGN PRINCIPLES?

- ✓ Provides guidelines for evaluating software structure
- ✓ Ensures software is flexible, reusable, and maintainable
- ✓ Covers behavior modeling using:
  - ➢ UML State Diagrams
  - ➢ UML Sequence Diagrams



2. EVALUATING DESIGN COMPLEXITY

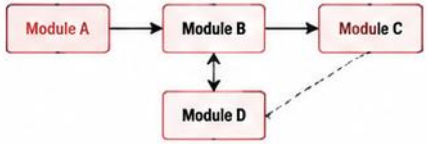
- ✓ Keep modules simple to avoid unnecessary complexity
- ✓ Excessive complexity → harder to understand → more bugs
- ✓ Evaluation of design complexity is essential for quality assurance
- ✓ Complexity applies to:
  - ➢ Classes
  - ➢ Methods within classes
- ✓ In this module, "module" = program unit containing classes and methods
- ✓ System = combination of modules



3. GOOD DESIGN vs BAD DESIGN (MODULE CONNECTIONS)

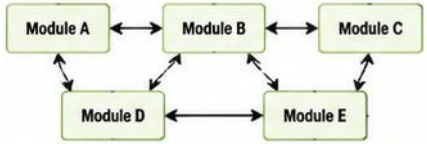
Bad Design:

- Modules can connect only to specific modules
- Limits flexibility and reuse



Good Design:

- Modules are compatible with each other
- Can be easily connected and re-used



4. COUPLING (Interdependence Between Modules)

Definition: Measures complexity between a module and other modules

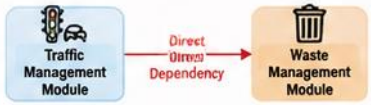
Types of Coupling:

- ➢ **Tight Coupling** (Bad Design) – module heavily depends on others
- ➢ **Loose Coupling** (Good Design) – module connects via well-defined interfaces

4A. TIGHT vs LOOSE COUPLING EXAMPLE (SMART CITY)

Tight Coupling (Bad Design / High Coupling)

Traffic Management Module directly depends on Waste Management Module to update road availability.



- If the waste system fails, the traffic system also fails.
- This makes the system rigid and hard to maintain.

VS

Loose Coupling (Good Design / Low Coupling)

Traffic Management Module reads from a shared Service (e.g., City Resource Service) for road availability.



- If waste management fails, traffic still works with last data.
- Modules can evolve independently and be reused in other cities.

4B. METRICS TO EVALUATE COUPLING

Degree – Number of connections
- Fewer connections → looser coupling

Ease – Clarity of connections
- Should not require understanding internal details of other modules

Flexibility – Interchangeability
- Modules should be easily replaceable

4C. SIGNS OF TIGHT COUPLING (BAD DESIGN)

- ⚠ Too many parameters or interfaces in connections
- ⚠ Difficulty finding corresponding modules
- ⚠ Module can only connect to specific modules (not interchangeable)

4C. COHESION (How Well a Module Works Internally)

Definition: Focuses on complexity within a module

Types of Cohesion:

- ➢ High Cohesion (Good Design) – module has a clear, single purpose
- ➢ Low Cohesion (Bad Design) – module has multiple or unclear responsibilities

5A. HIGH COHESION (GOOD DESIGN)

Water Supply Module
- Pipeline Monitoring
- Supply Scheduling
- Water Quality Checking
- Billing

- ✓ Preforms one task only (Water related)
- ✓ Purpose is clear and easy to understand
- ✓ Easy to maintain and extend
- ✓ (e.g., adding smart water sensors)

5B. LOW COHESION (BAD DESIGN)

Water Supply Module
- Water Distribution
- Traffic Control
- Garbage Collection

- ✗ Multiple unrelated tasks in one module
- ✗ Purpose is unclear → harder to maintain
- ✗ Encapsulation must be broken to understand

6. BALANCING COHESION & COUPLING

Goal: Low Coupling + High Cohesion

Trade-offs in Complex Systems:

- ➢ Increasing cohesion may increase coupling
- ➢ Reducing coupling may reduce cohesion



Find the Right Balance

7. EXAMPLE SUMMARY

Bad Design (Low Cohesion & Tight Coupling)



- ✗ Modules depend on each other
- ✗ Mixed responsibilities
- ✗ Hard to test, maintain, and reuse
- ✗ System becomes rigid

Good Design (High Cohesion & Loose Coupling)



- ✓ Modules are independent
- ✓ Single responsibility
- ✓ Easy to maintain and extend
- ✓ System is flexible and reusable

Key Takeaway: Design good software by keeping modules highly cohesive and loosely coupled. This leads to maintainable, flexible, reusable, and reliable systems.

System Design & Module Interaction

- ✓ **System = combination of modules**
- ✓ **Bad design**:
  - ✓ Modules can connect only to specific modules
  - ✓ Limits flexibility and reuse
- ✓ **Good design**:
  - ✓ Modules are **compatible** with each other
  - ✓ Can be **easily connected** and **re-used**
- ✓ **Evaluating Design Complexity**
  - ➢ **Coupling** (Interdependence between modules)– degree of interdependence between modules
  - ➢ **Cohesion** (How well a single module works internally)– degree to which elements within a module work together

Coupling in Software Design

- ✓ **Definition**: Measures complexity between a module and other modules
- ✓ **Types of Coupling**:
  - ➢ **Tight Coupling (Bad Design)** – module heavily depends on others
  - ➢ **Loose Coupling (Good Design)** – module connects via well-defined interfaces
- ◆ **Metrics to Evaluate Coupling**
- ✓ **Degree** – number of connections
  - ✓ Fewer connections → **looser coupling**
- ✓ **Ease** – clarity of connections
  - ✓ Should not require understanding internal details of other modules
- ✓ **Flexibility** – interchangeability
  - ✓ Modules should be **easily replaceable**
- ✓ **Signs of Tight Coupling (Bad Design)**
- ✓ Too many parameters or interfaces in connections
- ✓ Difficulty finding corresponding modules
- ✓ Module can only connect to **specific modules** (not interchangeable)

Coupling (Interdependence between modules)

- ✓ **Tight Coupling (Bad Design/ High Coupling):**
  Suppose the **Traffic Management Module** directly depends on the **Waste Management Module** to update road availability. For example, traffic routes can only be updated if the waste collection module first finishes its update.
  - ➢ If the waste system fails, the traffic system also fails.
  - ➢ This makes the system rigid and hard to maintain.

- ✓ **Loose Coupling (Good Design/Low Coupling):**
  The **Traffic Management Module** just reads from a **shared database or service** (e.g., a central "City Resource Service") that contains road availability. The Waste Management Module also updates the same database, but they don't directly depend on each other.
  - ➢ If waste management fails, traffic still works with the last available data.
  - ➢ Modules can evolve independently and be reused in other cities.

Cohesion in Software Design

- ✓ **Definition**: Focuses on complexity **within a module**
- ✓ **Types of Cohesion**:
  - ✓ **High Cohesion (Good Design)** – module has a **clear, single purpose**
  - ✓ **Low Cohesion (Bad Design)** – module has **multiple or unclear responsibilities**
- ✓ **Characteristics**
  - ✓ **High Cohesion**:
    - ➢ Module performs **one task only**
    - ➢ Purpose is **clear and easy to understand**
  - ✓ **Low Cohesion**:
    - ➢ Module performs **multiple unrelated tasks**
    - ➢ Purpose is unclear → harder to maintain
    - ➢ Encapsulation must be broken to understand

Cohesion in Software Design

- ✓ **Balancing Cohesion & Coupling**
  - ➢ **Goal: Low Coupling + High Cohesion**
  - ➢ **Trade-offs in Complex Systems**:
    - ➢ Increasing **cohesion** may increase **coupling**
    - ➢ Reducing **coupling** may reduce **cohesion**

Cohesion in Software Design

- ✓ **Bad Design (Low Cohesion):**
  The **Water Supply Module** not only distributes water but also contains code for traffic control and garbage collection.
  - ➢ Responsibilities are mixed.
  - ➢ Hard to test, maintain, or reuse.
- ✓ **Good Design (High Cohesion):**
  The **Water Supply Module** only handles tasks related to water: pipeline monitoring, supply scheduling, water quality checking, and billing.
  - ➢ Single responsibility.
  - ➢ Easy to maintain and extend (e.g., adding smart water sensors).

Coupling Cohesion Example-1

Scenario-1: Suppose we have a class called FarmController in the Automated Agricultural System that has two purposes: controlling irrigation and fertilizer dispensing. Here we have a controlSystem() method that takes a zero (0) flag if you want to activate the irrigation system, and takes the one (1) flag if you want to activate the fertilizer system. The class diagram for the FarmController and implementation of its controlSystem() method is as below:

FarmController
+ controlSystem(controlFlag: int)
	public void controlSystem(int controlFlag) {
	switch (controlFlag) {
	case 0:
	return this.startIrrigation();
	case 1:
	return this.startFertilizerDispense();
	default:
	throw new
	UnknownControlFlagException();
	}}

Problem a): Evaluate the above design in terms of **coupling** and **cohesion**. Give justification in each case.

Problem b): Suggest a **better design** for the above class that meets acceptable **coupling** and **cohesion** requirements.

Coupling Cohesion Example-1

a) Evaluation of the Current Design

- **Cohesion: Low Cohesion (Poor Design)-** The FarmController class is responsible for **two unrelated tasks**:
  1. **starting irrigation**, and
  2. **starting fertilizer dispensing**.
- Both actions belong to different subsystems of the agricultural system, yet they are handled in a single method (controlSystem()), which switches behavior via flags. This leads to **low cohesion** because:
  - ✓ The class performs **multiple independent functions**, violating the *Single Responsibility Principle (SRP)*.
  - ✓ The controlSystem() method does **more than one job**, decided by a control flag (0 or 1).
  - ✓ If future systems (e.g., pest-control, greenhouse ventilation) are added, the class must be modified repeatedly, further reducing cohesion.
  - ✓ Thus, the class is **not focused on one responsibility**, making it difficult to maintain and extend.

- **Coupling: Higher Than Necessary (Flag-Based Coupling)-** While the class itself may not depend on many external components, the **internal coupling** is unnecessarily strong:
  - ✓ The class **knows the internal details** of both irrigation and fertilizer systems.
  - ✓ Changing the logic of either subsystem would force modifications inside FarmController.
  - ✓ Using numeric flags (0 and 1) introduces **implicit, error-prone coupling**—only the sender of the flag knows what the values mean.
  - ✓ External components that call controlSystem(flag) must also depend on the meaning of those numbers, creating a hidden dependency.

This design leads to **tighter coupling** and reduces flexibility.

Coupling Cohesion Example-1

b) Suggested Improved Design (Better Cohesion & Lower Coupling)

To achieve a clean design, the responsibilities should be **separated** into their own classes, and FarmController should act only as a **coordinator**.

❖ Better Design Structure

1. IrrigationController Class

- ✓ Handles all irrigation-related operations.
- ✓ Highly cohesive: one responsibility.

2. FertilizerController Class

- ✓ Responsible only for fertilizer dispensing.
- ✓ Highly cohesive.

3 FarmController Class

- ✓ Coordinates actions by calling the appropriate controller.
- ✓ Low coupling: interacts with controllers through clean method calls—not numeric flags.

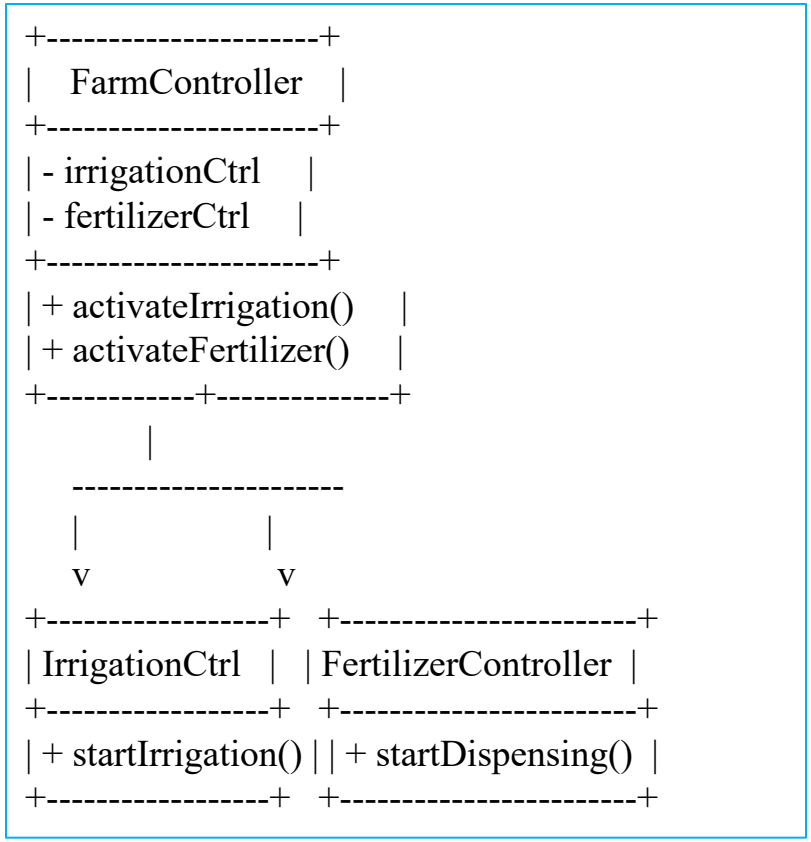


Figure: Improved UML-Style Diagram

Coupling Cohesion Example-1

Improved Code Example

```
public class FarmController {

    private IrrigationController irrigationCtrl;
    private FertilizerController fertilizerCtrl;

    public FarmController() {
        this.irrigationCtrl = new
IrrigationController();
        this.fertilizerCtrl = new FertilizerController();
    }

    public void activateIrrigation() {
        irrigationCtrl.startIrrigation();
    }

    public void activateFertilizer() {
        fertilizerCtrl.startDispensing();
    }
}
```

Figure: Improved Code

Why This Design Is Better

- **High Cohesion**
  - ✓ Each class has **one dedicated purpose**.
  - ✓ Easier to update and test.
- **Low Coupling**
  - ✓ FarmController does not know internal details of each subsystem.
  - ✓ New systems can be added (e.g., pestControl) by creating new classes without changing existing ones.
- **Readable and Maintainable**
  - ✓ Method names clearly describe actions.
  - ✓ Eliminates confusing flag-based logic.

Advantage of the code: No flags, No switch statements., Clear responsibilities.

Coupling Cohesion Example-2

Scenario-2: Suppose we have a class called ServiceManager in the Dhaka Urban Management System that has two purposes: handling waste collection requests and road maintenance requests.

Here we have a processRequest() method that takes a zero (0) flag if we want to process a waste collection request, and takes the one (1) flag if we want to process a road maintenance request. The class diagram for the ServiceManager and implementation of its processRequest() method is as below:

ServiceManager
+ processRequest(controlFlag: int)
	public void processRequest(int controlFlag) {
	switch (controlFlag) {
	case 0:
	return this.handleWasteCollection();
	case 1: return this.handleRoadMaintenance();
	default:
	throw new UnknownControlFlagException(); }}

a) Evaluate the above design in terms of **coupling** and **cohesion**. Give justification in each case.

b) Suggest a **better design** for the above class that meets acceptable **coupling** and **cohesion** requirements.

Separation of Concerns (SoC)

- ✓ One of the design principles examined in the previous module was that of decomposition.
- ✓ Decomposition divides a whole into different parts. To understand why decomposition is necessary in design, the principle of **separation of concerns** must be examined.

◆ **Separation of Concerns (SoC)**

- ✓ **Definition**: Dividing a system into distinct parts, each addressing a separate concern
- ✓ **Concern**: Anything that matters in solving a problem (e.g., data, logic, presentation)
- ✓ **Goal**: Keep concerns **separate and organized** in design and implementation

- ✓ A **concern** is a very general notion: it is anything that matters in providing a solution to a problem. Separation of concerns is about keeping the different concerns in your design separate.
- ✓ When software is designed, different concerns should be addressed in different portions of the software.

Separation of Concerns (SoC)

- One of the design principles examined in the previous module was that of **decomposition**.
- Decomposition divides a whole into different parts. To understand why decomposition is necessary in design, the principle of **separation of concerns** must be examined.

What is Separation of Concerns (SoC)?

Definition: Dividing a system into distinct parts, each addressing a **separate concern**.

Concern: Anything that matters in solving a problem (e.g., **data, logic, presentation**).

Goal: Keep concerns separate and organized in design and **implementation**.

Why is Separation of Concerns Important?

- A **concern** is a very general notion: it is anything that matters in providing a solution to a problem. Separation of concerns is about keeping the different concerns in your design separate.
- When software is designed, different concerns should be addressed in different portions of the software.
- Separation of concerns provides many advantages. They allow us to develop and update sections of the software **independently**.
- Using separation of concerns also means that we **do not need to know** how all sections of code work in order to update a section.
- Finally, separation of concerns allows changes to be made to one component **without requiring a change** in another.
- When addressing concerns separately, **more cohesive classes** are created and the design principles of **abstraction, encapsulation, decomposition,** and **generalization** are enforced.

SoC in Action: Without vs With Separation of Concerns

Without SoC (Tangled Design)

All concerns are mixed in one module.
Hard to understand, maintain, and extend.

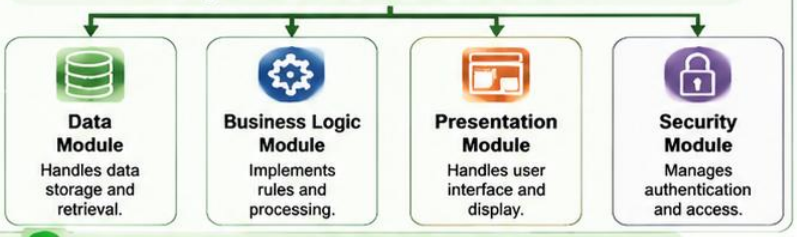


Any change affects the whole system.

VS

With SoC (Modular Design)

Each concern is separated into its own module.
Easy to understand, maintain, and extend.



Changes in one module do not affect others.

Advantages of Separation of Concerns

- Independent development and updates
- No need to know entire system to update one section
- Changes in one component do not force changes in others
- Improves maintainability, modularity, and reusability

How SoC Supports Key Design Principles

- **Abstraction**: Each concept is separated with its own relevant attributes and behaviors.
- **Encapsulation**: Attributes and behaviors are bundled in a class. Implementation can change without affecting the interface.
- **Decomposition**: A whole class can be broken down into multiple smaller classes.
- **Generalization**: Commonalities are identified and moved into a superclass.

SoC leads to clean design, fewer dependencies, easier maintenance, and better software quality.

Separation of Concerns (SoC)

- ✓ **Separation of concerns provides many advantages**. They allow us to develop and update sections of the software independently.
- ✓ Using separation of concerns also means that **we do not need to know how all sections of code work in order to update a section.**
- ✓ Finally, separation of concerns allows changes to be made to one component without requiring a change in another.
- ✓ When addressing concerns separately, more cohesive classes are created and the design principles of abstraction, encapsulation, decomposition, and generalization are enforced

Advantages of SoC

- ✓ **Abstraction occurs as each concept in the problem** space is separated with its own relevant attributes and behaviors.
- ✓ **Encapsulation occurs as the attributes** and behaviors are gathered into their own section of code called a class. Access to the class from the rest of the system and its implementation are separated, so details of implementation can change while the view through an interface can stay the same.

Separation of Concerns (SoC)

Advantages of SoC

- ✓ **Decomposition occurs** as a whole class can be separated into multiple classes.
- ✓ **Generalization occurs** as commonalities are recognized and subsequently separated and generalized into a superclass.
- ✓ Independent **development and updates** of software sections
- ✓ No need-to-know entire system to update one section
- ✓ **Changes in one component** do not force changes in others
- ✓ Improves **maintainability, modularity, and reusability**

Separation of Concerns Example

- ✓ **Consider a smartphone capable of many behaviors:**
  - taking photos,
  - scheduling meetings,
  - sending and receiving email,
  - browsing the Internet,
  - sending texts, and
  - making phone calls.
- ✓ This example will only focus on two functions **camera and traditional phone** functions.

Separation of Concerns Example

- ✓ This example will only focus on two functions camera and traditional phone functions.

```
public class SmartPhone {
private byte camera;
private byte phone;
public SmartPhone() { … }
public void takePhoto() { … }
public void savePhoto() { … }
public void cameraFlash() { … }
public void makePhoneCall() { … }
public void encryptOutgoingSound() { … }
public void decipherIncomingSound() { … } }
```

- ✓ This code has a **SmartPhone class with attributes called camera and phone**, and associated behaviors.
- ✓ This system has **low cohesion, as there are behaviors that are not related to each other**.
- ✓ The camera behaviors do not need to be encapsulated with the behaviors of the phone in order for the camera to do its job.
- ✓ The components also do not offer modularity.

Separation of Concerns Example

- ✓ For example, it is not possible to access the camera or the phone separately if another system is built that requires only one or the other.
- ✓ The camera also would not be able to be replaced with a different camera, or even a different object, without removing the code for the camera completely from this class.
- ✓ The SmartPhone class needs to be more cohesive, and each component of the smartphone should have distinctive functionality.
- ✓ **Using the separation of concerns, we can identify that the SmartPhone class has two concerns:**
  - ➢ To act as a traditional telephone.
  - ➢ To take pictures using the built-in camera.
- ✓ With the concerns identified, it is possible to separate them into their own more cohesive classes and encapsulate all the details about each concern into functionally distinct and independent classes.

Separation of Concerns Example

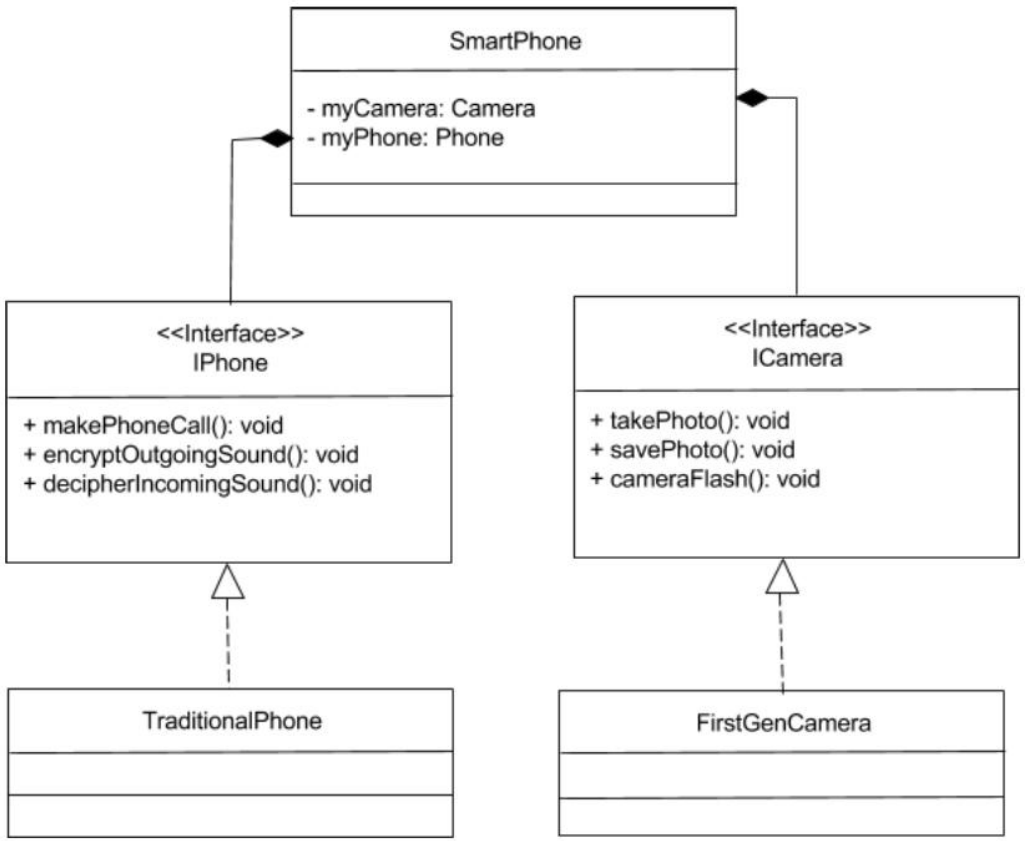
- ✓ **Problem: Low Cohesion in SmartPhone Class**
  - SmartPhone class mixes **camera** and **phone** functions
  - **Low cohesion**: unrelated behaviors grouped together
  - **No modularity**: cannot reuse/replace camera or phone independently

- ✓ **Solution: Separation of Concerns**
  - Identify two distinct concerns:
    - ✓ **Phone functionality**
    - ✓ **Camera functionality**
  - Create **separate cohesive classes**: Phone and Camera
  - SmartPhone acts as a **coordinator**, referencing both
  - Improves:
    - ✓ **Cohesion** (focused responsibilities)
    - ✓ **Modularity** (components can be reused/replaced)

Separation of Concerns Example

- ✓ The SmartPhone class will reference instances of the new classes so that the smartphone can act as a coordinator of the camera and the phone.
- ✓ This will let our smartphone provide access to all the behaviours of the camera and the phone, without having to know how each component behaves.
- ✓ Using a UML Class diagram, this is how the new design for the SmartPhone system might look:

Example: Smartphone System



The code would translate as below:

```
public interface ICamera {
public void takePhoto();
public void savePhoto();
public void cameraFlash();
}
public interface IPhone {
public void makePhoneCall();
public void encryptOutgoingSound();
public void deciphereIncomingSound();
}
public class FirstGenCamera implements
ICamera {
/* Abstracted camera attributes */
public class TraditionalPhone implements
IPhone {
/* Abstracted phone attributes */
}
```

- ✓ The attributes and behaviors for the phone and camera have been separated into two distinct interfaces. These are each implemented with a corresponding class. The code would translate as below:

Example: Smartphone System

The SmartPhone class also need to be redesigned for two separate classes:

```
public class SmartPhone {
private ICamera myCamera;
private IPhone myPhone;
public SmartPhone( ICamera aCamera, IPhone
                                aPhone ) {
                                this.myCamera = aCamera;
                                this.myPhone = aPhone;
                                }
public void useCamera() {
return this.myCamera.takePhoto();
}
public void usePhone() {
return this.myPhone.makePhoneCall();
}
}
```

Example: Smartphone System

- ✓ **Redesigned SmartPhone System**
  - SmartPhone aggregates **Phone** and **Camera** classes
  - **Constructor** takes IPhone and ICamera objects as parameters
  - SmartPhone forwards responsibilities to these objects
  - **Who creates the Phone/Camera objects** → separate responsibility
- ✓ **Benefits of Redesign**
  - **Modular design** – phone/camera can be swapped without changing SmartPhone code
  - **High cohesion** – Phone and Camera have clear, independent functionality
  - **Flexibility** – supports future upgrades (e.g., new camera type)
- ✓ **Trade-off**
  - Increased **coupling**: SmartPhone depends on IPhone and ICamera interfaces
- ✓ **Use of Separation of Concerns**
  - **Generalization** – define IPhone and ICamera interfaces
  - **Abstraction & Encapsulation** – implement separate Phone and Camera classes
  - **Decomposition** – split smartphone into distinct, reusable parts

Example: Smartphone System

- ✓ **SmartPhone Class Redesign**
  - **Constructor**Integrates **camera** and **phone** functionalities.
  - Keeps **Camera** and **Phone** classes separated (hidden from each other).
  - Both are aggregated under the **SmartPhone** class.
- ✓ **Design**
  - SmartPhone constructor accepts a **Camera** and a **Phone** as parameters.
  - New SmartPhone instances can be created by passing in objects implementing **ICamera** and **IPhone** interfaces.
- ✓ **Responsibility Separation**
  - Creation of appropriate **Camera** and **Phone** objects is handled outside the SmartPhone class.
  - SmartPhone does not need to know *how* these objects are created.
- ✓ **Method Delegation**
  - SmartPhone methods **forward responsibilities** to the aggregated camera and phone objects.

Example: Smartphone System

- ✓ **Modular Design Benefits**
  - Camera or Phone classes can be **swapped** without modifying SmartPhone code.
  - Only the instantiation of SmartPhone and its parts needs to change.
- ✓ **Cohesion & Coupling**
  - SmartPhone class is now **more cohesive**.
  - However, coupling increases since SmartPhone depends on **Camera** and **Phone** interfaces (indirect dependency).
- ✓ **Separation of Concerns Applied**
  - **Generalization** → Separate notions of Camera and Phone via **interfaces**.
  - **Abstraction & Encapsulation** → Define first-gen Camera and traditional Phone as **implementing classes**.
  - **Decomposition** → Break down SmartPhone into **constituent parts** (camera + phone) separated from the whole.

Information Hiding in Software Design

- ✓ **Definition:** Modules only access the information needed to do their job
- ✓ **Purpose:** Hide details that may change (algorithms, data representations)
- ✓ Information hiding is commonly associated with sensitive data —the more sensitive the data, the more likely it should have limited access.
- ✓ Assumptions: Exposed via APIs & interfaces
- ➢ **Benefits of Information Hiding**
  - ➢ Developers can **work independently** on modules
  - ➢ Reduces dependencies on **implementation details**
  - ➢ **Flexibility**: implementation can change without affecting others
  - ➢ **Security**: sensitive data protected
  - ➢ **Reliability**: prevents unwanted changes to critical attributes
- ✓ **Relation to Encapsulation**
  - ✓ **Encapsulation**: bundles attributes & behaviors into a class
  - ✓ Provides access via **interfaces**
  - ✓ Hides **implementation details** behind method signatures
  - ✓ Allows internal changes without breaking external expectations

Information Hiding in Software Design

Definition: Modules only access the information needed to do their job.

Purpose: Hide details that may change (algorithms, data representations).

Information hiding is commonly associated with sensitive data — the more sensitive the data, the more likely it should have limited access.

Assumptions: Exposed via APIs & interfaces

Benefits of Information Hiding

Developers can work **independently** on modules.

Reduces dependencies on **implementation details.**

Flexibility: implementation can change without affecting others.

Security: sensitive data protected.

Reliability: prevents unwanted changes to critical attributes.

Relation to Encapsulation

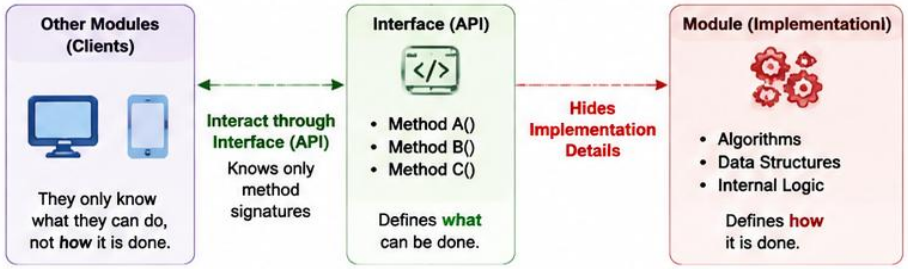
Encapsulation: bundles attributes & behaviors into a class.

Provides access via **interfaces.**

Hides **implementation details** behind method signatures.

Allows internal **changes** without breaking external expectations.

How Information Hiding Works



Key Ideas & Benefits

- **Purpose:** Conceals changeable details (e.g., algorithms, data structures) to reduce dependency.
- **Assumptions:** Not hidden—explicitly expressed in APIs and interfaces.
- **Key Idea:**
  - Hide what may change → implementation details.
  - Reveal what remains stable → assumptions via interfaces.
- **Benefits:**
  - Enables independent module development.
  - Promotes modularity and maintainability.
- **Relation to Encapsulation:**
  - Encapsulation groups atributes and behaviors into classes.
  - Interfaces provide controlled access, hiding internal implementations.
  - Other modules depend on method signatures, not underlying logic.

Encapsulation & Interfaces

Implementation is hidden behind an interface.

Other classes rely on method signatures, not internal logic.

Behaviour can change without altering expected outcomes.

Why It Matters

Expectations for a behaviour are fulfilled without exposing ***how*** it works.

Enhances flexibility, maintainability, and modularity.

Protects system from unintended interference.

Hiding Attributes

Critical attributes should not be modified directly by external classes.

Protects class integrity and prevents unintended side effects.

Access Modifiers (Control Visibility in Java)

Modifier	Where Accessible	Attributes (Fields)	Methods	Use / Notes
public	Any class in the system	Can be read / modified	Can be invoked	Widest access. Encapsulation still applies.
protected	Same package & subclasses (in any package)	Can be read / modified	Can be invoked	Useful for inheritance and related groups.
default (no modifier)	Same package only	Can be read / modified	Can be invoked	Also called package-private.
private	Within the class only	Cannot be accessed directly	Cannot be invoked directly	Strongest restriction. Access via getters/setters if needed.

Example (Java)

```
public class Person {
    String name; // default access
}
```

- Person class is public
- **name** has default access (no modifier)
- Could also be declared **private** or protected for stricter control

Information Hiding leads to secure, flexible, and maintainable software by exposing only what is necessary.

Information Hiding in Software Design

- ✓ **Purpose**: Conceals changeable details (e.g., algorithms, data structures) to reduce dependency.
- ✓ **Assumptions**: Not hidden—explicitly expressed in APIs and interfaces.
- ✓ **Key Idea**:
  - ➢ Hide what may change → implementation details.
  - ➢ Reveal what remains stable → assumptions via interfaces.
- ✓ **Benefits**:
  - ➢ Enables independent module development.
  - ➢ Promotes modularity and maintainability.
- ✓ **Relation to Encapsulation**:
  - ➢ Encapsulation groups attributes and behaviors into classes.
  - ➢ Interfaces provide controlled access, hiding internal implementations.
  - ➢ Other modules depend on method signatures, not underlying logic.

Information Hiding in Software Design

- ✓ **Encapsulation & Interfaces**
  - ➢ Implementation is hidden behind an interface.
  - ➢ Other classes rely on method signatures, not internal logic.
  - ➢ Behaviour can change without altering expected outcomes.
- ✓ **Why It Matters**
  - ➢ Expectations for a behaviour are fulfilled without exposing *how* it works.
  - ➢ Enhances flexibility, maintainability, and modularity.
- ✓ **Hiding Attributes**
  - ➢ Critical attributes should not be modified directly by external classes.
  - ➢ Protects class integrity and prevents unintended side effects.
- ✓ **Access Modifiers**
  - ➢ Control visibility of attributes and behaviours.
  - ➢ Define what subclasses and other classes can access.
  - ➢ Examples: public, private, protected.

Attributes & Access Modifiers (Java)

- ✓ Attributes can be **hidden** to prevent direct external modification
- ✓ **Access levels**:
  - ➢ **Public** – Accessible everywhere
  - ➢ **Protected** – Accessible within package & subclasses
  - ➢ **Default** – Accessible within package only
  - ➢ **Private** – Accessible within the class only
- ✓ **Public**
  - ➢ Accessible by **any class** in the system
  - ➢ **Attributes:** Can be read/modified by other classes
  - ➢ **Methods:** Can be **invoked** by other classes
  - ➢ **Encapsulation still applies** → Implementation hidden
- ✓ **Protected**
  - ➢ Accessible by:
    - ➢ The class itself
    - ➢ **Subclasses**
    - ➢ **Same package** classes
  - ➢ Useful for inheritance and related groups of classes

Attributes & Access Modifiers (Java)

- ✓ **Default (No Modifier)**
  - Also called **package-private**
  - Accessible only within the **same package**
  - Not visible to subclasses outside the package
- ✓ **Private**
  - Accessible **only within the class itself**
  - Attributes cannot be accessed directly by other classes
  - Methods cannot be invoked by external classes

```
public class Person {
    String name; // default access }
```

- Person class is public
- name has default access (no modifier)
- Could also be declared private or protected for stricter control

Conceptual Integrity in Software Design

- ✓ **Ensures a software system feels consistent and cohesive, as if guided by a single mind.**
- ✓ **Maintains uniformity in design and implementation, even with multiple contributors.**
- ✓ **How It's Achieved**:
  - ➢ Clear design principles and conventions.
  - ➢ Effective communication among team members.
  - ➢ Regular code reviews for consistency.
  - ➢ Unified architecture and design foundation.
  - ➢ Adoption of common programming constructs.
  - ➢ A small core group approving commits to ensure coherence.
- ✓ **Benefit**:
  - ➢ Produces software that is predictable, maintainable, and easier to understand.

Conceptual Integrity in Software Design

A software system should feel consistent and cohesive, as if guided by a single mind.

1. WHAT IS CONCEPTUAL INTEGRITY?

- ✓ Ensures software is **consistent** and **cohesive**.
- ✓ Makes the system feel like it was built by **one guiding mind**.
- ✓ Achieved through **shared design principles & conventions.**
- ✓ Not about ignoring **team opinions** → ideas must **align with agreed principles.**
- ✓ Promotes **uniform design** and implementation across thie system.

Benefit:
Produces software that is predictable, maintainable, and easier to understand.

2. WAYS TO ACHIEVE CONCEPTUAL INTEGRITY

- Effective **communication** among team members (Daily stand-ups, sprint retrospectives)
- **Code reviews** for consistency
- Use of **standard design principles** & constructs (Java Interfaces, Design Patterns)
- Adoption of a well-defined architecture (Design for single process, Architecture for multi-process)
- **Unifying concepts** across modules (Find commonality, reduce special cases)
- A small **core review group** to approve commits and ensure coherence

3. WHY IT MATTERS (EXAMPLES)

Dhaka Urban Management System

- **Without conceptual integrity:** traffic control, waste management, and safety systems might be built by different teams with different standards → leading to incompatibility and confusion.
- **With conceptual integrity:** all subsystems follow the same design principles, communication standards, and interfaces → citizens experience **one cohesive urban management platform.**

Agricultural Automation System

- **Without conceptual integrity:** irrigation, drones, and greenhouses might work as separate tools → farmers must learn and manage each independently.
- **With conceptual integrity:** tools share a common architecture (e.g., one dashboard, unified data standards) → system feels **seamless and farmer-friendly.**

4. KEY PRACTICES THAT PROMOTE CONCEPTUAL INTEGRITY

4.1 Code Reviews

- Systematic review of written code (like academic peer review).
- Detects mistakes and improves consistency among developers.
- Ensures adherence to agreed design principles.

4.2 Design Principles & Constructs



- **Java Interfaces** enforce consistent behaviors across classes.
- **Design Patterns** provide conventional, reusable solutions.
- Promote **uniformity** and conceptual integrity.

4.3 Well-Defined Design & Architecture



- **Design** guides internal structure of single-process software.
- **Architecture** describes multi-process interactions and relationships.
- Both provide consistent structure and logic.

4.4 Unifying Concepts

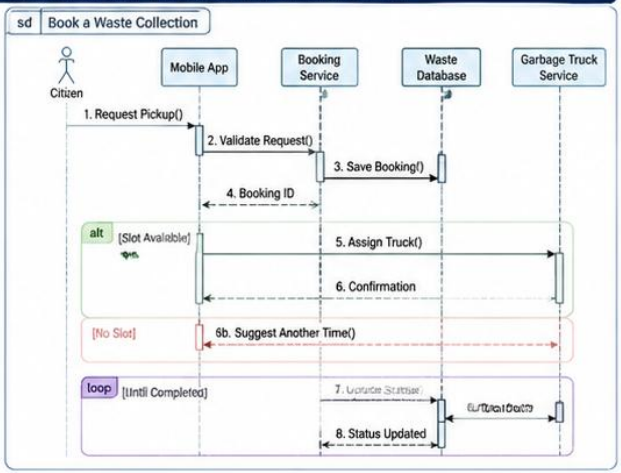
- Finding commonality across different resources/features.
- Example: Unix philosophy — "treat everything as a file".
- Simplifies usage and reduces special cases → more consistency.

4.5 Small Core Review Group

- A trusted **core team** accepts commits.
- Ensures alignment with architecture & design rules.
- Prevents unorganized or "informal" coding practices.

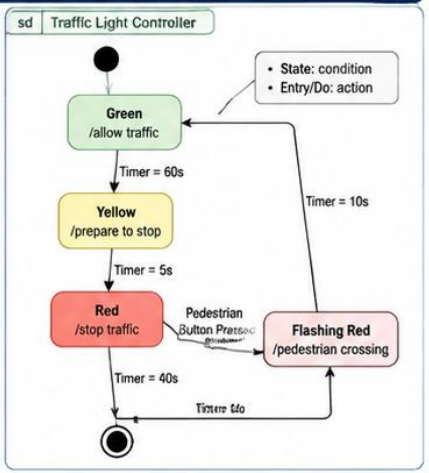
5. UML SEQUENCE DIAGRAM – HOW OBJECTS INTERACT OVER TIME

- **Objects / Roles** (Boxes at top)
- **Lifelines** (Vertical dashed lines)
- **Messages** (Solid arrows)
- **Return Messages** (Dashed arrows)
- **Activation Bars** (Thin rectangles)
- **Actors** (Stick figures)
- **Frame (Diagram box)** Title at top-left
- **Advanced Elements** alt (Alternative)
- **loop** (Loop)



6. UML STATE DIAGRAM – HOW AN OBJECT BEHAVES OVER TIME

- **States** (Rounded rectangles) Condition of an object at a point in time.
- **Transitions (Arrows)** Movement from one state to another.
- **Events (Triggers)** Cause a transition between states.
- **Actions (Outputs)** What the system does during transition or inside a state.
- **Initial State** (Black filled circle) Starting point.
- **Final State** (Circle with dot) End point.



Conceptual Integrity in Software Design

- ✓ Ensures software is **consistent and cohesive**
- ✓ Makes the system feel like it was built by **one guiding mind**
- ✓ Achieved through **shared design principles & conventions**
- ✓ Not about ignoring team opinions → ideas must align with agreed principles
- ✓ Promotes **uniform design and implementation** across the system

- ✓ **Ways to Achieve Conceptual Integrity**
  - ➢ **Effective communication** among team members
  - ➢ **Code reviews** for consistency
  - ➢ Use of **standard design principles & constructs**
  - ➢ Adoption of a **well-defined architecture**
  - ➢ **Unifying concepts** across modules
  - ➢ A **core review group** to approve commits

- ✓ Best Practice
  - ➢ Agile practices: **daily stand-ups** & **sprint retrospectives**
  - ➢ Agree on **libraries, frameworks, and methods** to ensure uniformity

Conceptual Integrity in Software Design

- ✓ **Dhaka Urban Management System & Conceptual Integrity**
  - If traffic control, waste management, safety, and citizen services are developed by different teams, the system could feel fragmented.
  - **Without conceptual integrity: traffic control, waste management, and safety systems might be built by different teams with different standards** → leading to incompatibility and confusion.
  - **With conceptual integrity: all subsystems follow the same design principles, communication standards, and interfaces** → citizens experience one **cohesive urban management platform**.
- ✓ **Agricultural Automation System & Conceptual Integrity**
  - Different modules (irrigation sensors, drones, greenhouses, AI models) are often developed separately.
  - **Without conceptual integrity: irrigation, drones, and greenhouses might work as separate tools** → farmers must learn and manage each independently.
  - **With conceptual integrity: these tools share a common architecture (e.g., one dashboard, unified data standards), making the system feel seamless** and farmer-friendly.
  - **Conceptual integrity ensures that complex systems—whether in urban management or agriculture—work as a unified whole, not just a collection of disconnected parts.**

Conceptual Integrity – Key Practices

- ✓ **Code Reviews**
  - ➢ Systematic review of written code (like academic peer review)
  - ➢ Detects mistakes and improves consistency among developers
  - ➢ Ensures adherence to agreed design principles
- ✓ **Design Principles & Constructs**
  - ➢ **Java Interfaces**: enforce consistent behaviors across classes
  - ➢ **Design Patterns**: conventional, reusable solutions to design problems
  - ➢ Promote uniformity and conceptual integrity

- ✓ **Well-Defined Design & Architecture**
  - ➢ **Design**: guides internal structure of single-process software
  - ➢ **Architecture**: describes multi-process interactions and relationships
  - ➢ Both provide consistent structure and logic
- ✓ **Unifying Concepts**
  - ✓ Finding commonality across different resources or features
  - ✓ Example: **Unix philosophy** – treat everything as a file
  - ✓ Simplifies usage and reduces special cases → more consistency

Conceptual Integrity – Key Practices On DUMS

1. Code Reviews

- ✓ Developers working on **traffic monitoring** and **waste collection modules** review each other's code.
- ✓ Detects logic errors (e.g., wrong sensor data handling).
- ✓ Ensures both modules follow the same **naming conventions, error handling, and security standards**.

2. Design Principles & Constructs

- ✓ **Java Interfaces**: A standard SensorData interface defines how all sensors (traffic, pollution, waste bins) send data.
- ✓ **Design Patterns**: Observer pattern used so multiple subsystems (traffic control, pollution alerts) can react to real-time sensor updates in the same way.
- ✓ Promotes **uniformity** across modules.

3. Well-Defined Design & Architecture

- ✓ **Design (internal)**: Traffic light control software has a clear internal structure separating data collection, decision logic, and output commands.
- ✓ **Architecture (multi-process)**: Traffic control, waste management, and emergency response services communicate through a **central IoT hub**.
- ✓ Provides a **consistent structure** for scaling and integration.

Conceptual Integrity – Key Practices On DUMS

4. Unifying Concepts

- ✓ **Single Dashboard**: Citizens and administrators use one portal for traffic updates, garbage collection schedules, and safety alerts.
- ✓ Like the **Unix philosophy**: "treat everything as a file," here → "treat all services as a data stream."
- ✓ Reduces special cases → improves **user experience and system consistency**.

- ✓ **Small Core Review Group**
  - ➢ A trusted core team accepts commits
  - ➢ Ensures alignment with architecture & design rules
  - ➢ Prevents unorganized or "informal" coding practices

- ✓ So, **conceptual integrity in Dhaka Urban Management System** ensures all modules—built by different teams—work as one cohesive, predictable, and maintainable system.

Specialized UML class diagrams

- ✓ Beyond standard UML class diagrams, this section explores two specialized types and their application in technical design. UML **sequence diagrams, UML State Diagram.**
- ✓ UML **sequence diagrams** describe how objects interact with each other over time to complete a task. They are widely used in **planning and technical design**, before coding begins.
- ✓ Think of a sequence diagram as a **conversation map**: it shows the messages exchanged between objects (or actors) to achieve a goal.

◆ Key Components of a Sequence Diagram

- ✓ **Objects / Roles (Boxes at the top)**
  - ➢ Represent roles played by objects (often named after their class).
- ✓ **Lifelines (Vertical dashed lines)**
  - ➢ Show the passage of time for an object.
- ✓ **Messages (Solid arrows)**
  - ➢ Represent communication from one object to another.
  - ➢ Label above arrow describes the message/action.

Specialized UML class diagrams

- ✓ **Return messages (Dashed arrows)**
  - ➢ Show data/control returning to the sender.
- ✓ **Activation bars (Small rectangles on lifeline)**
  - ➢ Indicate when an object is active (sending/receiving/waiting).
- ✓ **Actors (Stick figures)**
  - ➢ Represent people interacting with the system.
- ✓ **Frame (Large box)**
  - ➢ Encloses the diagram; title placed at the top-left.
- ✓ **Advanced elements**
  - ➢ **alt** (Alternative): shows conditional flows.
  - ➢ **loop**: repeats actions while a condition holds true.

Example: Changing TV Channel with a Remote

- ✓ **Simple Sequence:**
  - Actor (**Viewer**) sends input (press button) → **Remote**.
  - **Remote** forwards command → **TV**.
  - **TV** changes channel → displays to Viewer.
- ✓ **Extended with Alternatives & Loops:**
  - If Viewer is **not satisfied** with the channel → loop repeats.
  - Viewer presses "up/down" → Remote → TV → New channel.
  - Continues until Viewer is satisfied.

UML State Diagrams

- ✓ A **UML State Diagram** describes how a system or an object **behaves and responds** to events over time. It shows the **different states** an object can be in, and the **transitions** between those states when events occur.
- ✓ A **state** represents the condition of an object at a specific moment, defined by its **attributes and behavior**.

◆ **Key Elements of a State Diagram**

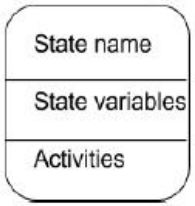
- ✓ **States (Rounded rectangles)**
  - ✓ Show the condition of an object at a point in time.
  - ✓ Example: *Park, Reverse, Neutral, Drive* in a car.
- ✓ **Transitions (Arrows)**
  - ✓ Show the movement from one state to another when an event occurs.
- ✓ **Events (Triggers)**
  - ✓ Cause a transition between states.
  - ✓ Example: *Shift gear, Press start, Turn off*.
- ✓ **Actions (Outputs/Behaviors)**
  - ✓ What the system does during a transition or inside a state.
- ✓ **Initial State (Black filled circle)**
  - ✓ The starting point of the system/object.
- ✓ **Final State (Circle with dot inside):** Where the system/object ends.

UML State Diagrams

- A filled circle indicates the starting state of the object. Every state diagram begins with a filled circle.



- Rounded rectangles indicate other states. These rectangles have three sections: a state name, state variables, and **activities**.



  - State names should be short, meaningful titles for the state of the object. Each state should have at least a state name.

- Arrows indicate **transitions** from one state to another. Transitions are typically triggered by an event. This event is generally described above the arrow.



  Each transition arrow will always have an event, and it may even have a guard condition and an action. The transition and action happens from a given state if the event occurs and the condition is true.
- A circle with a filled circle inside indicates termination. Termination represents an object being destroyed or the process being completed. Not all diagrams have a termination–some may run continuously.

  - State variables are data relevant to the state of the object.
  - **Activities** are actions that are performed when in a certain state. There are three types of activities



present for each state: Entry, Exit, and Do activities. **Entry activities** are actions that occur when the state is just entered from another state. **Exit activities** are actions that occur when the state is exited and moves on to another state. **Do activities** are actions that occur while the object is in a certain state.

Example: Car with Automatic Transmission

Example: Car with Automatic Transmission

✓ A car can exist in **different states**:

- **Park** → Car is stationary and locked.
- **Reverse** → Car moves backward.
- **Neutral** → Car wheels are disengaged.
- **Drive** → Car moves forward.

Transition Example:

- If the car is in **Reverse**, it can only move backward.
- To move forward, the driver triggers a **gear shift event**, changing the state to **Drive**.

✓ **Why Use State Diagrams?**

- ✓ **Clarify behavior**: Easier to see how an object responds to different inputs than reading source code.
- ✓ **Reveal missing cases**: Helps identify unplanned conditions or transitions.
- ✓ **Support testing**: Knowing all possible states ensures **complete test coverage**.
- ✓ **Better communication**: Provides a shared visual language for designers, developers, and testers.

Model Checking

- ✓ Model checking is a **formal verification technique** used to systematically check whether a software system behaves correctly in **all its possible states**.
  - ➢ It explores every possible state transition of the system.
  - ➢ It helps identify errors that might not be caught by traditional testing methods (unit tests, beta testing, simulations, etc.).

◇ **How It Works**

1.Generate a State Model

1. From your software (e.g., code or specification), the model checker builds a **state model**.
2. A state model is like an abstract state machine that represents the system's states and transitions.

2.Check Behavioural Properties

1. The model checker verifies whether the state model satisfies specific properties, such as:
   1. No **deadlock** (system should not freeze with tasks waiting for each other).
   2. No **race conditions** (two processes accessing shared resources unpredictably).
   3. Certain events must always eventually occur.

Model Checking

1. ◇ **Why It's Useful Explore All States**
    1. The model checker simulates every possible state change caused by different events and inputs.
    2. If a violation is found (e.g., a deadlock is possible), the tool provides a **counterexample trace** showing how it could happen.

✓ **Example**

- Suppose a system has a **rule: "No deadlock should occur."**
- The model checker simulates all possible state transitions.
- If it finds that Process A and Process B could wait forever for the same resource, it reports a **deadlock violation**.

✓ Finds errors not easily detected by testing.
✓ Ensures system reliability, especially in safety-critical systems (banking, aviation, medical devices, etc.).
✓ Complements other testing techniques.

Example: Smartphone System

- ✓ Initial Design:
  - Single SmartPhone class with both camera and phone functions
  - Low cohesion → unrelated behaviors mixed together
  - Poor modularity → cannot reuse or replace components separately
- ✓ Improved Design (with SoC):
  - Identify two concerns:
    - Phone functionality
    - Camera functionality
  - Create separate cohesive classes (Phone, Camera)
  - SmartPhone acts as coordinator, referencing these components
  - Provides access without needing internal details

Thank You (Q/A)



